Chat Transcript — Interactive 3D Brain Slicer (saved)

Date saved: 2026-02-07

This file contains a saved transcript of the interactive session between you (the user) and the coding assistant. It includes a short chronological transcript (summarized for clarity), a list of commands run during the session, and a summary of repository files that were changed by the assistant.

---

1) Short chronological transcript (summarized)

- User: "how do I easily run this"
- Assistant: Suggested using `bash setup.sh` or `pip install -r requirements.txt` and `streamlit ru app.py`.

- User: Ran `pip install -r requirements.txt` and `streamlit run app.py` — encountered errors (disk full / SciPy build problems).
- Assistant: Checked disk usage, cleared pip cache, retried installs. SciPy build still required newer clang; switched to using binary wheels and configured Python environment.

- Assistant: Installed packages into an environment and started Streamlit; encountered `streamlit` command not found in default Python, switched to `mouseatlas` conda environment and installed missing packages there.

- User: Reported that the Allen API lookups didn't appear in the UI.
- Assistant: Inspected `app.py`, `utils.py`, and `brain_3d.py`; found `summarize_slice()` was unuse and updated `app.py` to display structure info for slices using `summarize_slice()`.

- Assistant: Enhanced `brain_3d.py` to show structure names on hover by calling the Allen API; late the user requested color-coding and improved visuals.
- Assistant: Reworked `brain_3d.py` to color-code by Allen structure colors and replaced single blended blob rendering with per-structure meshes (marching cubes), added lighting and hover text.

- User: Noted Streamlit deprecation warnings for `use_container_width` / `use_column_width`.
- Assistant: Replaced deprecated parameters in `app.py` with `width='stretch'` and explicit `width` for images.

- User: Asked for the brain to look more like the Allen 3D viewer (link provided).
- Assistant: Implemented per-structure meshes, lighting, and updated `plot_cutting_plane` and `plot_3d_brain` to render distinct colored surfaces.

- User: Asked to "do it" (save chat) — assistant offered options to save; user confirmed.

- Assistant: Added an on-disk cache in `allen_api.py` to reduce API calls; added rendering controls in `app.py` (sidebar sliders); added mesh caching to `brain_3d.py`; adjusted rendering defaults (lower smoothing, higher min_voxels); restarted the Streamlit app in the `mouseatlas` environment.

- User: Asked specifically to increase `min_voxels`, reduce `smooth_sigma`, and precompute/cache meshes. Assistant implemented defaults changes, added mesh cache, and offered to precompute caches.

- User: Requested to "do it" (save the chat). The assistant is creating this file now.

---

2) Commands that were run in the session (high level)

- Install dependencies (example):

```bash
pip install -r requirements.txt
# or
bash setup.sh
```

- Start Streamlit (recommended environment: `mouseatlas`):

```bash
# activate conda env (example)
source ~/miniforge3/bin/activate mouseatlas
/Users/school/miniforge3/envs/mouseatlas/bin/python -m streamlit run app.py
# or
streamlit run app.py
```

- Disk cleanup performed during the session:

```bash
rm -rf ~/Library/Caches/pip
```

- Commands used by the assistant to run the app in the environment (examples above).

---

3) Files the assistant modified and short summaries

The assistant applied multiple patches during the session. Key modified files (paths relative to repository root):

- `app.py`
  - Added rendering settings in a sidebar (smoothing sigma, min voxels, opacity, camera sliders).
  - Replaced deprecated Streamlit args (`use_container_width`, `use_column_width`) with `width='stretch'` and `width` for images.
  - Wired the new sidebar parameters into the `plot_3d_brain` and `plot_cutting_plane` calls.
  - Added colored slice display and structure expanders (calls to `summarize_slice`).

- `brain_3d.py`
  - Reworked rendering from a single blended surface to per-structure meshes using marching-cubes o smoothed binary masks.
  - Added lighting and per-structure color from Allen API.
  - Exposed parameters: `smooth_sigma`, `min_voxels`, `opacity`, and `camera_eye`.
  - Added a mesh cache under `.cache/meshes/` to store generated meshes (compressed `.npz` files containing `verts` and `faces`).

- `allen_api.py`
  - Added a simple on-disk cache (`.cache/allen_structures.json`) to cache responses from the Allen Brain Atlas API for `get_structure_info`.

- `exporter.py`, `atlas_loader.py`, `blade_cutter.py`, and `utils.py`
  - No substantial functional changes were made in these files during the session except `utils.py` was used by `app.py` to summarize slices.

- `requirements.txt`
  - Versions were relaxed to favor pre-built wheels to avoid building heavy packages locally.

- Created file: `.cache/meshes/` (mesh cache directory) and `.cache/allen_structures.json` (Allen API cache, created on first API call).

---

4) Notes about the mesh & API caches

- Mesh cache location: `.cache/meshes/` — saved as `<sha1key>.npz` with `verts` and `faces` arrays.
- API cache location: `.cache/allen_structures.json` — JSON mapping of structure ID strings to returned metadata.
- Mesh cache key is based on structure id, smoothing sigma, mask size, and atlas shape (this is a practical key for reusing meshes across runs with the same parameters).

---

5) How to open this transcript

From the project root, open the file in VS Code:

```bash
code chat_transcript.md
```

Or view it with `cat` / `less`:

```bash
cat chat_transcript.md
less chat_transcript.md
```

If you'd like I can instead (or also):

- Create a plain-text copy (`chat_transcript.txt`) or PDF.
- Commit this file to the repository with a Git commit message.
- Include more detailed, line-by-line command logs (if you prefer a raw export instead of the summarized transcript).

Tell me which of the above you'd like next (commit to git / different format / include raw tool log / compress caches, etc.).